PropBench: A Benchmark for Engineering Judgment in Change Propagation

Aakash Sangwan
Independent Researcher
aakashsangwan024@gmail.com | github.com/Aakash2408/Propbench

Abstract

When engineers modify an API, they must identify all downstream consumers that will break - a task requiring architectural knowledge, codebase familiarity, and pattern recognition. We introduce PropBench, a benchmark of 268 real-world change propagation scenarios from 24 repositories (industrial and open-source), annotated with ground-truth consequences totaling 1,223 consequence files. We evaluate automated baselines and find that engineering propagation judgment decomposes into three largely independent channels: naming conventions (7% file recall), co-change history (17% file recall with fair cross-validation, 38% upper bound with full training), and domain-specific patterns. An ensemble combining all channels achieves 82% package-level recall. Our miss classification analysis reveals that 39% of propagation targets are cross-package, requiring knowledge beyond single-file analysis. PropBench enables reproducible comparison of change propagation tools and establishes that the majority of senior engineering judgment in this task is decomposable into learnable, repo-specific patterns.

Keywords: change propagation, software engineering, benchmark, developer tools, co-change analysis

1. Introduction

Software systems evolve through coordinated changes across multiple packages. When a developer modifies an API contract - adding a required field, removing a deprecated endpoint, changing a message schema - downstream consumers must be updated or they will fail silently in production. Our observations across 18 repositories show that propagating a single API change typically involves 2-3 days of coordination, 3-6 engineers, and touches 2-4 packages.

We ask: how much of the engineering judgment involved in predicting change propagation can be captured by automated tools? Prior work has focused on detecting API breaking changes (Optic, buf) or managing library versions (Dependabot, Renovate). These solve detection but not propagation.

1.1 Contributions

1. PropBench - a benchmark of 268 annotated change propagation scenarios with ground-truth consequences, spanning 5 contract types across 24 repositories.
2. Decomposition analysis - engineering judgment decomposes into three independent channels achieving 82% package-level recall.
3. Miss classification - analysis of 1,223 consequence files revealing cross-package dependencies (39%) and non-conventional naming (26%) as primary barriers.

2. Related Work

Ren et al. (2004) introduced Chianti for change impact analysis in Java. Lehnert (2011) surveyed change impact analysis techniques. Rolfsnes et al. (2016) demonstrated generalizing from co-change history. Tools like Optic (OpenAPI), buf (Protobuf), and GraphQL Inspector detect breaking changes. Dependabot and Renovate automate version bumping but do not propagate contract changes. Our work measures the propagation step: given a known change, predict the full set of downstream files requiring modification.

3. Dataset Construction

Data sources: 241 industrial entries from 18 repos (git-mined), 16 expert-curated entries from 6 repos, and 11 open-source entries from 5 repos (FastAPI, React, Kubernetes, Django, Next.js). Total: 268 entries across 24 repositories.

3.1 Difficulty Classification

Trivial: 45 (17%) - Direct dependency, any tool catches
Easy: 72 (27%) - Clear pattern, straightforward
Medium: 89 (33%) - Requires pattern recognition or history
Hard: 48 (18%) - Requires architectural knowledge
Expert: 14 (5%) - Requires tribal/runtime knowledge

3.2 Contract Types

OpenAPI: 98 entries (6 breaking change types)
Protobuf: 62 entries (6 types)
GraphQL: 38 entries (6 types)
Database: 45 entries (6 types)
AsyncAPI: 25 entries (6 types)

4. Methodology

4.1 Baselines

FilePredictor: Naming conventions only. 7% file recall, 16% precision.
Historian (5-fold CV): Co-change frequency, fair estimate. 17% file recall, 17% precision.
Historian (full training): Upper bound with complete history. 38% file recall, 43% precision.
Ensemble (package-level): Weighted combination of all channels. 82% package recall.

The gap between FilePredictor (7%) and Historian-CV (17%) demonstrates that co-change learning adds +10 percentage points even with limited history. The gap to Historian-full (38%) shows prediction improves with more commit history - the system gets smarter with more data.

4.2 Evaluation Protocol

We use 5-fold cross-validation for the Historian baseline to prevent data leakage: the model is trained on 4 folds and tested on the held-out fold. The "full training" result (38%) represents an upper bound achievable with complete repository history.

4.3 Miss Classification

For each consequence file that FilePredictor misses (93% of all files):
- Test files: 34% - Naming does not follow 1:1 convention
- Same-package (non-test): 26% - No naming relationship to trigger
- Config/YAML/JSON: 16% - Domain knowledge required
- CDK/Infrastructure: 10% - Architectural coupling
- Model/Schema: 10% - Schema-implementation mapping
- Generated code: 2% - Partially detectable
- Documentation: 1% - Partially detectable
- Proto registry: 1% - Domain-specific

5. Results

5.1 Decomposability Thesis

The three channels are largely independent: naming catches test files (34% of consequences by category), history catches same-package co-changes (+10% unique over naming), patterns catch config + CDK + domain rules. Combined package-level recall of 82% with minimal overlap confirms decomposability.

5.2 The Learning Curve

The most significant finding is the gap between naming-only (7%) and history-trained (17%-38%) prediction. This demonstrates: (1) naming conventions alone are fundamentally insufficient, (2) repository-specific learning is the primary value driver (+10% with limited history, +31% with full history), (3) more data = better predictions.

5.3 Cross-Package Challenge

39% of consequence files are in different packages than the trigger. This cross-package propagation is invisible to any single-repository analysis tool, requiring either cross-repository co-change mining, explicit dependency graph traversal, or organizational knowledge about package relationships.

6. Discussion

Change propagation tools should not rely solely on naming conventions or grep. An effective tool must combine: (1) convention-based prediction (fast, high precision for test files), (2) co-change learning (adapts to codebase-specific patterns over time), (3) domain playbooks (encodes organizational knowledge).

The finding that the majority of propagation is predictable suggests that the "expert judgment" engineers spend days on is largely pattern-matching, not genuine reasoning. This time could be automated, saving 2-3 days per API change propagation event.

6.1 Limitations

- Industrial data cannot be shared publicly (reproducibility limited to OSS subset)
- Human baselines collection is in progress (N < 15)
- Single organization for industrial data
- File-level scoring for PatternOracle/StructureOracle requires further development

7. Conclusion

We introduced PropBench, a benchmark for measuring engineering judgment in change propagation. Our analysis of 268 scenarios with 1,223 consequence files shows that propagation knowledge decomposes into three independent channels. A co-change learning baseline achieves 17% file recall with fair cross-validation (vs. 7% for naming alone), with an upper bound of 38% given full history. At the package level, an ensemble achieves 82% recall. These results suggest that this fundamental engineering task - which currently costs organizations days of manual coordination per change - is largely automatable through repository-specific learning.

References

1. Ren, X., Shah, F., Tip, F., Ryder, B.G., & Chesley, O. (2004). Chianti: A tool for change impact analysis of Java programs. OOPSLA.

2. Lehnert, S. (2011). A taxonomy for software change impact analysis. IWPSE-EVOL.

3. Rolfsnes, T., Moonen, L., Di Alesio, S., Behjati, R., & Binkley, D. (2016). Generalizing the analysis of evolutionary coupling. SANER.

4. GitHub. (2019). Dependabot: Automated dependency updates.

5. Optic. (2021). OpenAPI breaking change detection.
6. buf. (2020). Protobuf breaking change linting.